THE GUSTA | GUIA PASSO A PASSO

Cadastro de cliente com API (Criar conta)

Etapa 1 — Importar AsyncStorage e API

No início de src/app/cad-cliente.tsx, acrescente os imports abaixo.

```
import AsyncStorage from "@react-native-async-storage/async-storage";
import { API } from "@/config/api";
```

Etapa 2 — Criar os estados do formulário

Dentro de CadClienteScreen(), mantenha os estados existentes (verSenha, verConfirmarSenha, aceitarTermos e modalTermosUso) e acrescente:

```
const [nome, setNome] = useState("");
const [email, setEmail] = useState("");
const [telefone, setTelefone] = useState("");
const [senha, setSenha] = useState("");
const [confirmarSenha, setConfirmarSenha] = useState("");
const [carregando, setCarregando] = useState(false);
const [mensagem, setMensagem] = useState("");
```

Cada estado guarda o valor de um campo.

“carregando” evita envios repetidos;

“mensagem” apresenta os erros na própria tela.

Etapa 3 — Vincular os cinco TextInput

Em cada TextInput adicione as propriedades indicadas, mantendo as existentes:

```
// Nome
value={nome}
onChangeText={setNome}

// E-mail
value={email}
onChangeText={setEmail}

// Telefone
value={telefone}
onChangeText={setTelefone}

// Senha
value={senha}
onChangeText={setSenha}

// Confirmar senha
value={confirmarSenha}
onChangeText={setConfirmarSenha}
```

Os botões para mostrar e esconder senhas e o checkbox de termos continuam iguais.

Etapa 4 — Identificar o dispositivo

Antes da função cadastrar(), dentro do componente, reutilize o mesmo padrão do Login. O identificador fica salvo em “device_id”.

```
async function nomeDoDispositivo() {
  let id = await AsyncStorage.getItem("device_id");
  if (!id) {
    id = `app-${Date.now()}-${Math.random().toString(36).slice(2, 8)}`;
    await AsyncStorage.setItem("device_id", id);
  }
  return id;
}
```

Etapa 5 — Criar a função cadastrar()

Coloque a função a seguir antes do return do componente. Primeiro são verificadas as entradas e o aceite dos termos; depois é enviado o POST para a API.

```
async function cadastrar() {
  if (!nome.trim() || !email.trim() || !telefone.trim() || !senha || !confirmarSenha) {
    setMensagem("Preencha todos os campos.");
    return;
  }

  if (senha.length < 8) {
    setMensagem("A senha deve ter pelo menos 8 caracteres.");
    return;
  }

  if (senha !== confirmarSenha) {
    setMensagem("As senhas não correspondem.");
    return;
  }

  if (!aceitarTermos) {
    setMensagem("Aceite os termos de uso para continuar.");
    return;
  }

  setCarregando(true);
  setMensagem("");

  try {
    const resposta = await fetch(`${API}/auth/cadastro`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Accept: "application/json",
      },
      body: JSON.stringify({
        nome: nome.trim(),
        email: email.trim(),
        telefone: telefone.trim(),
        senha: senha,
        aceite_termo: aceitarTermos,
        device_name: await nomeDoDispositivo(),
      }),
    });
```

```
    const json = await resposta.json();

    if (!resposta.ok || !json.success) {
      setMensagem(json.message || "Erro ao cadastrar cliente.");
      return;
    }

    await AsyncStorage.setItem("token", json.data.token);
    await AsyncStorage.setItem(
      "cliente", JSON.stringify(json.data.cliente)
    );

    router.replace("/home");
  } catch (erro) {
    console.log("Erro ao cadastrar cliente:", erro);
    setMensagem("Não foi possível conectar ao servidor.");
  } finally {
    setCarregando(false);
  }
}
```

Nessa etapa não existirá token pois o cadastro é público.

Etapa 6 — Exibir mensagens de validação

Antes do botão “Criar conta”, insira o componente condicional. Sem este Text, setMensagem() atualiza o estado, mas nada aparece na interface.

```
{mensagem ? (
  <Text style={cadClienteStyle.erroCadastro}>
    {mensagem}
  </Text>
) : null}
```

Em src/styles/cadClienteStyle.ts, dentro do StyleSheet.create(), acrescente:

```
erroCadastro: {
  color: "#CC0000",
  fontSize: 14,
  textAlign: "center",
  marginBottom: 12,
},
```

Etapa 7 — Conectar o botão Criar conta

Substitua o onPress que apenas navegava para o Login pela chamada de cadastrar(). Mantenha os estilos originais.

```
<Pressable
  style={({ pressed }) => [
    cadClienteStyle.btnCriar,
    pressed && cadClienteStyle.btnCriarPressed,
  ]}
  onPress={cadastrar}
  disabled={carregando}
>
  <Text style={cadClienteStyle.textoCriar}>
    {carregando ? "Cadastrando..." : "Criar conta"}
  </Text>
</Pressable>
```

Etapa 8 — Integração com o Perfil: data de nascimento opcional

A API de cadastro não exige data de nascimento, enquanto no perfil, existe a necessidade de data de nascimento. Portanto, um cliente recém-cadastrado pode ter data_nasc_cliente igual a null. O cadastro permanece inalterado: a correção é realizada em src/app/perfil.tsx.

8.1 — Erro ao abrir o perfil

Ao abrir o Perfil, o aplicativo tenta executar .substring() sobre data_nasc_cliente mesmo quando o valor é null. Isso interrompe a renderização da tela.

8.2 — Corrigir o placeholder do TextInput

No TextInput da data de nascimento, substitua a expressão de placeholder pela verificação abaixo:

```
placeholder={
  cliente?.data_nasc_cliente
    ? cliente.data_nasc_cliente
        .substring(0, 10)
        .split("-")
        .reverse()
        .join("/")
    : "Data de nascimento"
}
```

Se houver data, ela aparece no formato DD/MM/AAAA. Caso contrário, o campo mostra “Data de nascimento”, sem gerar erro.

8.3 — Verificar o carregamento dos dados

O trecho de carregarCliente() já verifica se a data existe. Mantenha essa condição:

```
if (json.data.data_nasc_cliente) {
  const dataAPI = json.data.data_nasc_cliente.substring(0, 10);
  const dataBR = dataAPI.split("-").reverse().join("/");
  setDataNascimento(dataBR);
}
```